

MCDIA500 · PROGRAMACIÓN PARA LA CIENCIA DE DATOS

AVANCE DEL PROYECTO · FASES 1 Y 2

**Duración de la titulación en el pregrado universitario
chileno (2025)**

Integrantes (Grupo 6): Fernanda Ovalle Román - Sebastián Cajales Cid - César Lorca Bacián – Jorge Álvarez Ossandón

Curso: MCDIA500 · Programación para la Ciencia de Datos · Magíster en Ciencia de Datos e Inteligencia Artificial

Docente responsable: Dr. Omar Salinas Silva

Institución: Universidad Andrés Bello · UNAB Online

Fecha: 13 de septiembre de 2026

Repositorio: <https://github.com/Feroroman/proyecto-grupo6-mcdi500>

ÍNDICE

I. Portada e índice	1
II. Introducción y contextualización	3
III. Definición de la problemática y objetivos del proyecto	4
3.1 Formulación de la problemática y preguntas centrales	4
3.2 Objetivo general y objetivos específicos	4
3.3 Alcance y supuestos	4
3.4 Articulación con las fases F1–F4	5
IV. Aplicación de herramientas científicas	5
4.1 Ecosistema científico de Python en el proyecto	5
4.2 Conjunto de datos y sus variables	6
4.3 Reproducibilidad técnica	6
4.4 Control de versiones	7
4.5 Documentación del proceso	8
4.6 Repositorio GitHub	9
V. Diseño de soluciones algorítmicas eficientes	9
5.A Codificación funcional	9
5.B Preprocesamiento y transformación del dataset	10
5.C Validación técnica y verificación del código	12
VI. Notebook F1_Definicion.ipynb (F1)	13
VII. Notebooks ejecutables (F2)	13
VIII. Vinculación del mapa conceptual con el avance del proyecto	14
IX. Bibliografía	15
Anexos técnicos	17
Anexo A · Bitácora de decisiones	17
Anexo B · Columnas del dataset procesado	17
Anexo C · Salida de las pruebas automatizadas	17
Anexo D · Notebooks ejecutados	18

II. Introducción y contextualización

2.1 Contexto y antecedentes

En Chile, la duración real de las carreras universitarias suele superar su duración formal. Cada semestre adicional tiene consecuencias concretas: mayor costo para las familias y para el sistema de financiamiento, postergación del ingreso al mercado laboral y presión sobre la capacidad de las instituciones. El Ministerio de Educación publica cada año, a través de su portal de Datos Abiertos, el registro completo de personas tituladas de la educación superior, con información de la carrera, la institución, la sede, la jornada, la modalidad y la duración del estudio y del proceso de titulación (Ministerio de Educación de Chile, 2026).

2.2 Presentación del problema

No está claro qué características de la carrera, de la institución y del estudiante acompañan una titulación más larga. El proyecto busca medir esa asociación en la cohorte de titulados de 2025 del pregrado universitario, información relevante para quienes diseñan políticas de retención y titulación oportuna (Mineduc, CNED, vicerrectorías académicas).

2.3 Objetivo general y alcance

Caracterizar la duración de la titulación en el pregrado universitario chileno (cohorte 2025) e identificar los factores observables asociados a una mayor duración, mediante un flujo de trabajo reproducible en Python. Este informe cubre las Fases 1 y 2: la definición del problema, la selección y verificación del conjunto de datos, la configuración del entorno reproducible y colaborativo, y el pipeline de obtención, exploración, limpieza, transformación y validación de los datos. El análisis de asociación y su comunicación quedan para las Fases 3 y 4.

2.4 Justificación técnica

La problemática exige un enfoque programático por tres razones. Primero, el volumen: la base nacional tiene 328.998 registros y 40 columnas, inabordable con herramientas manuales. Segundo, la calidad: el registro contiene códigos de relleno (por ejemplo, el año 1900 como «sin información»), duplicados y variables categóricas que deben codificarse antes de cualquier análisis; cada una de esas decisiones debe quedar justificada y ser repetible, lo que solo garantiza un pipeline en código con control de versiones (Wilson et al., 2017; The Turing Way Community, 2022). Tercero, la trazabilidad: el trabajo es grupal y el historial de cambios debe mostrar quién hizo qué y por qué, lo que exige Git y GitHub como herramientas de trabajo (Chacon & Straub, 2014).

2.5 Conexión con las fases F1–F4

Fase	Aporte a la solución del problema	Estado en este informe
F1 · Definición	Pregunta analizable, dataset verificado, entorno reproducible, estructura del repositorio y mapa conceptual	Implementada (cap. III, IV, VI)
F2 · Datos	Pipeline que convierte el registro crudo en un dataset limpio, codificado y validado, con cada decisión justificada	Implementada (cap. V, VII)
F3 · Algoritmos	Funciones y clases para calcular y comparar la duración por grupos con criterios de eficiencia	Proyectada
F4 · Comunicación	Visualizaciones, informe final y presentación que respondan la pregunta	Proyectada

III. Definición de la problemática y objetivos del proyecto

3.1 Formulación de la problemática y preguntas centrales

Situación. La duración real de las carreras universitarias supera con frecuencia la formal.

Problema. No se conoce qué factores observables de la carrera, la institución y el estudiante se asocian a

una mayor duración hasta la obtención del título, medida en semestres (`dur_total_carr`).

Pregunta central. ¿Qué factores observables —tipo de universidad (CRUCH/privada), jornada, modalidad, área de conocimiento, región de la sede, sexo y rango etario— se asocian a un mayor número de semestres hasta la obtención del título entre los titulados de pregrado universitario de 2025?

Preguntas secundarias que orientan la búsqueda de evidencia:

- ¿Difiere la duración mediana entre jornada diurna, vespertina y a distancia, y se mantiene esa diferencia dentro de cada área de conocimiento?
- ¿Qué parte de la duración total corresponde al proceso de titulación (`dur_proceso_tit`) frente a los semestres de estudio?
- ¿Los registros sin año de ingreso original (código 1900) corresponden a trayectorias distintas, y cómo afecta su tratamiento a las conclusiones?

La pregunta central cumple las tres condiciones de analizabilidad: los datos necesarios existen en el conjunto elegido, los conceptos se miden con esas variables y el alcance está acotado.

3.2 Objetivo general y objetivos específicos

Objetivo general. Caracterizar la duración de la titulación en el pregrado universitario chileno (cohorte de titulados 2025) e identificar los factores observables asociados a una mayor duración, mediante un flujo de trabajo reproducible en Python.

Objetivos específicos.

- Obtener y verificar el conjunto de datos de titulados 2025 del Mineduc, documentando su origen, sus variables y sus problemas de calidad (F1).
- Configurar un entorno reproducible y un repositorio colaborativo con control de versiones (F1).
- Construir un pipeline de limpieza y transformación que trate valores faltantes, códigos de relleno, duplicados y variables categóricas, justificando cada decisión con cifras (F2).
- Validar el dataset resultante y las funciones del pipeline con comprobaciones automáticas y pruebas (F2).
- Implementar funciones y clases reutilizables para calcular y comparar la duración por grupos (F3).
- Comunicar los hallazgos con visualizaciones e informe técnico (F4).

3.3 Alcance y supuestos

- **Alcance de datos.** Se analiza solo el pregrado en universidades (105.063 registros); quedan fuera institutos profesionales, centros de formación técnica, postítulos y posgrados, cuya duración no es comparable.
- **Alcance técnico.** Análisis descriptivo y de asociación; no se construyen modelos predictivos ni se afirma causalidad.
- **Alcance temporal.** Cohorte de titulados 2025 únicamente; no se comparan cohortes.
- **Supuestos sobre las variables.** `dur_total_carr` (semestres desde el ingreso a la carrera hasta el título) es una medida válida de la duración; el código 1900 en el año de ingreso representa información no disponible y no un año real.
- **Supuestos sobre el dataset.** Los datos están anonimizados (identificador `mrn`) y son públicos; no se trabaja con datos personales identificables. Cada registro es un título, no una persona: una persona puede aparecer más de una vez.
- **Restricciones metodológicas.** La imputación por mediana de grupo y el escalamiento robusto se eligieron por su efecto medido sobre la dispersión; otros criterios podrían llevar a elecciones distintas.

3.4 Articulación con las fases F1–F4

Objetivo específico	Fase	Producto
---------------------	------	----------

Dataset verificado y documentado	F1	notebooks/F1, docs/validacion_subconjunto.md
Entorno reproducible y repositorio	F1	requirements.txt, README.md, .gitignore, historial de GitHub
Pipeline justificado	F2	notebooks/F2, src/, docs/bitacora.md
Validación y pruebas	F2	src/validacion.py, tests/, docs/verificacion.md
Funciones y clases para comparar grupos	F3	Proyectado
Visualización e informe final	F4	Proyectado

IV. Aplicación de herramientas científicas

4.1 Ecosistema científico de Python en el proyecto

El proyecto usa las cuatro piezas del ecosistema científico de Python, cada una con una función distinta (Kluyver et al., 2016; McKinney, 2010; Harris et al., 2020):

Herramienta	Función	Uso concreto en el proyecto
Python 3.9 + venv	Lenguaje y entorno aislado	Entorno .venv por integrante; kernel «Python (mcdi500)»
pandas 2.3.3	Tablas: lectura, filtrado, agregación	read_csv con sep «;», filtro del subconjunto, perfil de nulos, groupby, get_dummies, to_csv
NumPy 2.0.2	Cálculo numérico vectorizado	np.nan para faltantes, máscaras booleanas (logical_and), cálculo de escaladores sin bucles
JupyterLab 4.5	Documento ejecutable	notebooks/F1 y notebooks/F2: narrativa, código y salidas en orden
Matplotlib 3.9	Figuras	Diagramas de caja de atípicos y distribuciones bajo escaladores (docs/figuras/)
Git 2.x / GitHub	Historial con autoría y repositorio remoto	19 commits, convención tipo: descripción, resolución de un conflicto con git pull

La lógica está separada en módulos de src/ con funciones de parámetros explícitos, y los notebooks solo orquestan y documentan:

Módulo / script	Funciones	Etapas
src/proyecto.py	clase ProyectoF1: verificar_entorno, verificar_estructura, aplicar_filtro, generar_subconjunto, generar_readme	F1
src/validar_dataset.py	detectar_separador, validar (script del curso, con informe Markdown)	F1
src/exploracion.py	perfil_faltantes, atipicos_ric, resumen_categorias	F2 · paso 6
src/limpieza.py	eliminar_duplicados, codigo_a_nulo, eliminar_columnas_constantes, comparar_imputaciones, imputar_mediana	F2 · paso 7
src/transformacion.py	codificar_ordinal, one_hot, agrupar_raras, derivar_fechas, binaria_01	F2 · paso 8
src/escalado.py	escalar_estandar, escalar_minmax, escalar_robusto, comparar_escaladores	F2 · paso 9
src/validacion.py	validar_dataset_final, verificar_propiedad_escalador	F2 · paso 10
src/bitacora.py	registrar, exportar	F2 · paso 11

4.2 Conjunto de datos y sus variables

Fuente: «Titulados de Educación Superior 2025», Datos Abiertos Mineduc, Centro de Estudios (Ministerio de Educación de Chile, 2026). Archivo 20260817_Titulados_Ed_Superior_2025_WEB.csv, separador «;», UTF-8, 328.998 filas × 40 columnas, 175,6 MB. Son datos públicos del Estado de Chile, de acceso libre y

gratuito; el portal no declara una licencia Creative Commons específica, por lo que se reutilizan con atribución a la fuente conforme a la Ley 20.285 sobre acceso a la información pública. Como la base completa supera el límite de 100 MB de GitHub y mezcla niveles no comparables, el proyecto trabaja sobre un subconjunto derivado: pregrado en universidades, 105.063 filas, 55,8 MB, verificado con el validador del curso (docs/validacion_subconjunto.md).

Rol analítico	Variables del subconjunto	Tratamiento en la Fase 2
Identificador	mrún (104.595 únicos; 7 nulos)	Se conserva fuera del análisis
Númerica (semestres)	dur_estudio_carr, dur_proceso_tit, dur_total_carr	Atípicos por RIC; escalamiento robusto
Númerica discreta	anio_ing_carr_ori, anio_ing_carr_act, version	Código 1900 → faltante → imputación por grupo
Binaria	gen_alu (1/2)	Recodificada 0/1 (mujer); 56,3 % / 43,7 %
Nominal	tipo_inst_2, jornada, modalidad, area_conocimiento, region_sede, tipo_plan_carr	One-hot encoding (36 columnas)
Ordinal	rango_edad (6 niveles), nivel_carrera_1 (4)	Codificación con orden declarado
Fecha	fecha_obtencion_titulo (AAAAMMDD), fec_nac_alu (AAAAMM)	Parseo; mes de titulación y edad al titularse
Alta cardinalidad	nomb_carrera (1.096), nomb_inst (55), nomb_sede, comuna_sede	Agrupación de categorías con < 100 registros
Constantes	cat_periodo, tipo_inst_1, nivel_global	Eliminadas

4.3 Reproducibilidad técnica

Un trabajo es reproducible cuando otra persona, con los mismos datos y el mismo código, obtiene el mismo resultado (Wilson et al., 2017). El proyecto cumple las tres condiciones:

- **Entorno.** Entorno virtual creado con `python3 -m venv .venv` en cada computador; dependencias declaradas en `requirements.txt` generado con `pip freeze` e instaladas con `pip install -r requirements.txt`. Kernel registrado como «Python (mcdi500)»; el notebook F1 lo verifica con `verificar_entorno()` y falla si el intérprete no está dentro de `.venv`.
- **Código.** Ambos notebooks se ejecutan de principio a fin con Restart Kernel and Run All Cells (8 y 18 celdas de código). Rutas relativas a la raíz del repositorio, resuelta automáticamente. Evidencia: `docs/evidencias/*.pdf` (Anexo D) y `docs/verificacion.md`.
- **Datos.** El dato crudo (`data/raw/`) nunca se modifica ni se versiona; el README indica cómo descargarlo. Los derivados se regeneran con los notebooks en `data/processed/`. El repositorio versiona el código que produce los datos, no los datos.

Registro del entorno utilizado (pip freeze, macOS, 12-09-2026):

Componente	Versión	Componente	Versión
Python	3.9 (macOS, arm64)	JupyterLab	4.5.10
pandas	2.3.3	ipykernel	6.31.0
NumPy	2.0.2	nbstripout	0.8.2
Matplotlib	3.9.4	Git	2.x (Command Line Tools)

Instrucciones para reproducir desde cero (también en el README):

```
git clone https://github.com/Feroroman/proyecto-grupo6-mcdi500.git && cd proyecto-grupo6-mcdi500
python3 -m venv .venv && source .venv/bin/activate
python -m pip install -r requirements.txt
python -m ipykernel install --user --name mcdi500 --display-name "Python (mcdi500)"
# descargar el CSV original del portal Mineduc a data/raw/
python tests/test_proyecto.py && python tests/test_pipeline.py
```

```
jupyter lab # notebooks/F1/F1_Definicion.ipynb y luego notebooks/F2/F2_Pipeline.ipynb →
Restart Kernel and Run All Cells
```

4.4 Control de versiones

Git registra el historial local con autoría; GitHub aloja el repositorio remoto compartido (Chacon & Straub, 2014). Reglas del equipo: identidad configurada (user.name con nombre completo, user.email de la cuenta GitHub) antes del primer commit; mensajes con prefijo docs, data, feat, fix o test seguido de la descripción del cambio; nbstripout activo para limpiar salidas de notebooks antes de cada commit; .gitignore excluye el entorno virtual, todos los CSV, las salidas temporales y la configuración del IDE. Sobre ramas: en las Fases 1 y 2 el equipo trabajó sobre main repartiéndose por archivo, con un solo conflicto (resuelto con git pull, sin --force); para las Fases 3 y 4 se acordó una rama por integrante y fase (p. ej. f3-algoritmos-cesar) integrada por pull request.

Historial de la rama main (26 commits, orden cronológico):

Commit	Autor	Mensaje
083e518	Fernanda Ovalle Román	docs: crea estructura inicial del proyecto
7c52a0b	Fernanda Ovalle Román	feat: agrega validador del dataset y bitácora de decisiones
cfb2d41	Fernanda Ovalle Román	docs: define problema, pregunta y entorno reproducible (notebook F1)
5a6aa55	Fernanda Ovalle Román	docs: conserva la carpeta data/raw para el dato crudo
866d671	sebastiancajalescid	Fase 7 pipeline - criterio de procesamiento - código split ambos casos
b9b68b8	Fernanda Ovalle Román	feat: implementa módulos del pipeline en src/
1a7349b	Fernanda Ovalle Román	test: agrega pruebas de las funciones del pipeline
ec3e806	Fernanda Ovalle Román	feat: implementa pipeline F2 (exploración, limpieza, transformación, escalamiento, validación)
59ce7e1	Fernanda Ovalle Román	docs: reorganiza notebooks en carpetas F1 y F2 y actualiza README
39b4e47	Fernanda Ovalle Román	docs: agrega bitácora, figuras y notebooks ejecutados como evidencia
4ca6c2e	Fernanda Ovalle Román	fix: integra el borrador de exploración del equipo en notebooks/F2 y excluye .idea/ del versionado
461cdc6	Fernanda Ovalle Román	feat: encapsula la configuración de la Fase 1 en la clase ProyectoF1
ae4eb4d	Fernanda Ovalle Román	fix: compatibilidad del validador con Python 3.9
7b83d7c	Fernanda Ovalle Román	test: agrega pruebas de la clase ProyectoF1 y del validador
fd71e78	Fernanda Ovalle Román	docs: agrega registro de verificación y actualiza README y evidencias
342ae53	César Lorca Bacián	docs: registra ejecución verificada en macOS (César Lorca Bacián)
cb20755	César Lorca Bacián	docs: documenta entorno de ejecución en macOS
7bca968	César Lorca Bacián	test: documenta resultados de pruebas automatizadas
d2d96d4	César Lorca Bacián	docs: registra preparación de notebooks
27a8271	César Lorca Bacián	docs: agrega mapa mcdi500_s1_grupo6.pdf a la carpeta F1
0aa544a	Sebastian Cajales Cid	docs: elimina notebook F1 duplicado y referencia

		entregables e informe en README
e4ceb86	Sebastian Cajales Cid	docs: agrega borrador de informe editable en formato docx para revision del equipo
ad03375	Sebastian Cajales Cid	docs: Nombre de nuevo integrante añadido
e2bb5b1	Feroroman	Remove kernel restart instruction from notebook
11a4f0b	jaossandon	Corrección de informe realizado sobre 5 puntos pendientes
	jaossandon	Revisión y actualización de commits

4.5 Documentación del proceso

El equipo adoptó el nivel justificativo de documentación: registrar no solo qué se hizo sino por qué y qué alternativas se descartaron. Tres mecanismos lo sostienen: las celdas Markdown de los notebooks (explican cada decisión antes del código que la ejecuta), la bitácora (src/bitacora.py anota cada decisión con sus cifras y la exporta a docs/bitacora.md, 18 entradas, Anexo A) y el registro de verificación (docs/verificacion.md).

Fragmento de código comentado: imputación con bandera (src/limpieza.py)

```
def imputar_mediana(df, columna, grupo=None):
    """Imputa nulos con la mediana (global o por grupo) y crea la bandera <columna>_imputada."""
    out = df.copy() # nunca se modifica el DataFrame de entrada
    nulos = out[columna].isna() # máscara de los valores a imputar
    std_antes = out[columna].std() # dispersión antes, para medir el efecto
    if grupo: # mediana del grupo al que pertenece cada fila
        valores = out.groupby(grupo)[columna].transform("median").fillna(out[columna].median())
        out.loc[nulos, columna] = valores[nulos]
    else:
        out.loc[nulos, columna] = out[columna].median()
    out[f"{columna}_imputada"] = nulos.astype(int) # bandera: permite excluir imputados después
    cifras = {"n_imputados": int(nulos.sum()),
              "cambio_desv_pct": round(100 * (out[columna].std() - std_antes) / std_antes, 2)}
    return out, cifras # el resultado y la evidencia de la decisión
```

La función devuelve, además del DataFrame, las cifras que justifican la decisión; el notebook las pasa a la bitácora. Ese patrón (entrada explícita → salida nueva + cifras) se repite en todo src/.

Enfoques descartados y cambios justificados

Enfoque descartado	Por qué
Datos de una empresa y datos sintéticos (propuesta inicial)	Sin fuente verificable ni licencia; los datos simulados no permiten justificar el preprocesamiento (retroalimentación docente de la Formativa 1)
Versionar el subconjunto de 55 MB en data/raw/	Contradecía el mapa (data/raw = original intocable) y dejaba 55 MB en el historial; se reescribió el historial una única vez y ahora ningún CSV se versiona
Git LFS para subir el CSV original de 175 MB	Innecesario: el dato crudo se descarga del portal; el repositorio guarda el código que lo reproduce
Eliminar los registros con código 1900	No son aleatorios (duran 6 vs 10 semestres): eliminarlos sesga y pierde el 13 % de la muestra
Imputar con media o mediana global	Reducen la dispersión más de 6 %; la mediana por grupo la cambia +2,8 %
StandardScaler / MinMaxScaler	Incoherentes con la imputación por mediana y con el 9 % de atípicos; MinMax comprime el 90 % de los datos
Modelo de regresión, GitHub Actions y Docker (mapa inicial)	Fuera del alcance de las Fases 1–4; representar lo que no se implementará penaliza la vinculación con el mapa

4.6 Repositorio GitHub

Enlace: <https://github.com/Feroroman/proyecto-grupo6-mcdi500>

Repositorio público, rama main. Las carpetas de fase están identificadas como notebooks/F1 y notebooks/F2; el README técnico (generado desde el notebook F1) describe el proyecto, la estructura, las dependencias y las instrucciones de ejecución de cada notebook.

```
proyecto-grupo6-mcdi500/
├── data/raw/           dato crudo descargado del portal; no se modifica ni se versiona
├── data/processed/     derivados regenerables: subconjunto (F1) y dataset limpio (F2)
├── notebooks/F1/       F1_Definicion.ipynb
├── notebooks/F2/       F2_Pipeline.ipynb · F2_borrador_exploracion.ipynb (borrador del
equipo)
├── src/                proyecto · exploracion · limpieza · transformacion · escalado ·
validacion · bitacora · validar_dataset
├── tests/              test_proyecto.py (8) · test_pipeline.py (11)
├── docs/               validacion_subconjunto.md · bitacora.md · verificacion.md · figuras/
· evidencias/
└── requirements.txt · .gitignore · README.md
```

Contribuciones individuales (historial de main al 13-09-2026):

Integrante	Commits	Aportes
Fernanda Ovalle Román	14	Estructura, notebooks F1 y F2, módulos src/, pruebas, documentación, resolución del conflicto
Sebastián Cajales Cid		Borrador de exploración (paso 7) conservado en notebooks/F2/F2_borrador_exploracion.ipynb
César Lorca Bacián	5	Configuración del entorno en su equipo y verificación de ejecución; commit pendiente al cierre de este informe
Jorge Álvarez Ossandón	2	Corrección y actualización de informe y commits

El equipo declara este desbalance con transparencia: los tres integrantes tienen acceso de escritura y la convención de trabajo acordada para las Fases 3 y 4 (rama por integrante, pull requests) busca corregirlo.

V. Diseño de soluciones algorítmicas eficientes

5.A Codificación funcional

El pipeline está implementado como funciones puras con parámetros explícitos: cada una recibe un DataFrame (y los parámetros de la decisión), devuelve un DataFrame nuevo y las cifras que justifican el cambio, y nunca modifica la entrada. Las tareas están separadas por etapa en módulos independientes (exploración, limpieza, transformación, escalado, validación, bitácora), y el notebook F2 solo los orquesta. El flujo de datos es verificable porque cada etapa produce una variable distinta:

```
df0 (105.063 × 40) --eliminar_duplicados, eliminar_columnas_constantes, codigo_a_nulo-->
df1 (105.060 × 37)
df1 --imputar_mediana (anio_ing_carr_ori por grupo; sem_ing_carr_ori)--> df2 (105.060 ×
39)
df2 --codificar_ordinal, derivar_fechas--> df3 (105.060 × 43)
df3 --one_hot, agrupar_raras, binaria_01, escalar_robusto--> df4 (105.060 × 84)
df4 --validar_dataset_final (5 asserts)-->
data/processed/titulados_2025_pregrado_univ_limpio.csv
```

Eficiencia: todas las operaciones son vectorizadas con pandas/NumPy (máscaras booleanas, groupby().transform, get_dummies); no hay bucles sobre filas. El pipeline completo sobre 105.000 registros corre en menos de un minuto en un MacBook Air. Estilo: docstrings en cada función, tipos anotados, nombres en español coherentes con el dominio, y comentarios donde la intención no es evidente.

Fragmento comentado: verificación de la propiedad de cada escalador (src/validacion.py)

```
def verificar_propiedad_escalador(serie, escalador, tol=1e-6):
    """Comprueba la propiedad que garantiza cada escalador (validar es demostrar, no afirmar)."""
    if escalador == "estandar":
        # media 0 y desviación 1
        return abs(serie.mean()) < tol and abs(serie.std(ddof=0) - 1) < tol
    if escalador == "minmax":
        # rango [0, 1]
        return abs(serie.min()) < tol and abs(serie.max() - 1) < tol
    if escalador == "robusto":
        # mediana 0 y rango intercuartílico 1
        return abs(serie.median()) < tol and abs((serie.quantile(0.75) - serie.quantile(0.25)) -
1) < tol
    raise ValueError(f"Escalador desconocido: {escalador}") # entrada inválida → excepción
explicita
```

5.B Preprocesamiento y transformación del dataset

Exploración: medir antes de tocar nada

- **Faltantes.** nombre_grado 8,0 % y nombre_titulo 2,9 % (etiquetas de texto); cod_carrera 0,3 %, version 0,3 %, mrun 7 registros. Códigos de relleno: anio_ing_carr_ori = 1900 y sem_ing_carr_ori = 0 en 13.396 filas (12,8 %).
- **Hallazgo clave.** Los registros con código 1900 tienen duración mediana de 6 semestres frente a 10 en el resto, y su año de ingreso a la carrera actual coincide con el de origen en el 91 % de los casos válidos: el código no es un faltante aleatorio.
- **Categorías.** Sin variantes de escritura en las nominales; nomb_carrera con 1.096 valores, 968 de ellos con menos de 100 registros.
- **Atípicos.** dur_total_carr: mediana 10, Q1 = 8, Q3 = 10, RIC = 2; 9.845 atípicos (9,4 %) por 1,5-RIC; máximo 24 semestres (Figura 1).

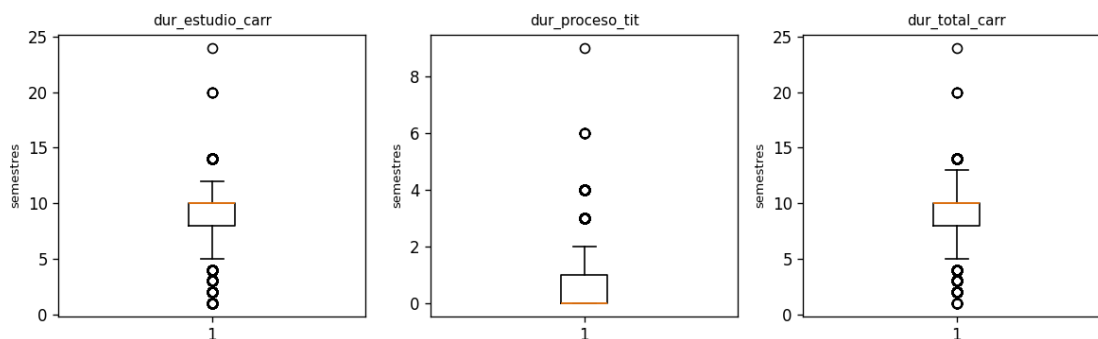


Figura 1. Diagramas de caja de las duraciones (semestres) antes de la limpieza.

Limpieza y manejo de valores faltantes

Decisión	Cifras	Justificación
Eliminar duplicados exactos	3 filas (0,003 %) → 105.060	Registros idénticos no aportan información y sesgarían conteos
Eliminar columnas constantes	cat_periodo, tipo_inst_1, nivel_global	Un solo valor tras el filtro de F1
Código 1900 y 0 → NA	13.396 filas (12,8 %)	1900 no es un año; tratarlo como número contaminaría cualquier cálculo
Imputar anio_ing_carr_ori con mediana por anio_ing_carr_act	13.396 imputados; desviación +2,8 %; bandera anio_ing_carr_ori_imputada	Comparación de cuatro estrategias (tabla siguiente): eliminar sesga; media y mediana global reducen la dispersión más de 6 %; la mediana por grupo usa una variable observada y es la que menos deforma
Imputar sem_ing_carr_ori con la mediana	13.396 imputados con 1	Variable 1/2; la mediana conserva la moda

Conservar NA en nombre_titulo, nombre_grado, mrun	2.997 · 8.434 · 7	Etiquetas e identificador: no son variables de análisis
---	-------------------	---

Estrategia para anio_ing_carr_ori	n	Media	Desv.	Cambio desv. %
Observados (referencia)	91.664	2019,449	2,561	0,00
Eliminar filas	91.664	2019,449	2,561	0,00 (pierde el 12,8 % de la muestra)
Imputar media	105.060	2019,449	2,392	-6,59
Imputar mediana	105.060	2019,519	2,399	-6,32
Imputar mediana por anio_ing_carr_act (elegida)	105.060	2019,812	2,633	+2,82

Transformaciones por tipo de variable (casting y codificación)

Rol	Variables	Transformación y justificación
Ordinal	rango_edad, nivel_carrera_1	Codificación entera con orden declarado por el equipo (15–19 < 20–24 < ... < 40 y más; Técnico < Licenciatura no conducente < Profesional sin licenciatura < Profesional con licenciatura). Dejar que la librería ordene alfabéticamente produciría una escala inventada
Nominal	tipo_inst_2, jornada, modalidad, area_conocimiento, region_sede	One-hot encoding → 36 columnas 0/1; sin orden natural, cada categoría es una dimensión
Alta cardinalidad	nomb_carrera	968 categorías con < 100 registros agrupadas en OTRA → 129 categorías; evita más de mil columnas casi vacías
Fecha	fecha_obtencion_titulo, fec_nac_alu	Casting de entero a fecha (formatos AAAAMMDD y AAAAMM); derivadas mes_titulacion y edad_titulacion (mediana 25,6 años; rango 19,2–80,9)
Binaria	gen_alu (1/2)	Recodificada a mujer (0/1); 56,3 % mujeres
Numéricas	dur_total_carr, dur_estudio_carr, edad_titulacion	Normalización con RobustScaler (mediana 0, RIC 1), coherente con la imputación por mediana y con el 9 % de atípicos; MinMax comprimiría el 90 % de los datos (RIC 0,087) por el máximo de 24 semestres

Escalador (dur_total_carr)	Media	Desv.	Mediana	RIC	Mín	Máx
Original	9,074	2,440	10,000	2,000	1,000	24,000
Estándar	0,000	1,000	0,380	0,820	-3,309	6,116
MinMax	0,351	0,106	0,391	0,087	0,000	1,000
Robusto (elegido)	-0,463	1,220	0,000	1,000	-4,500	7,000

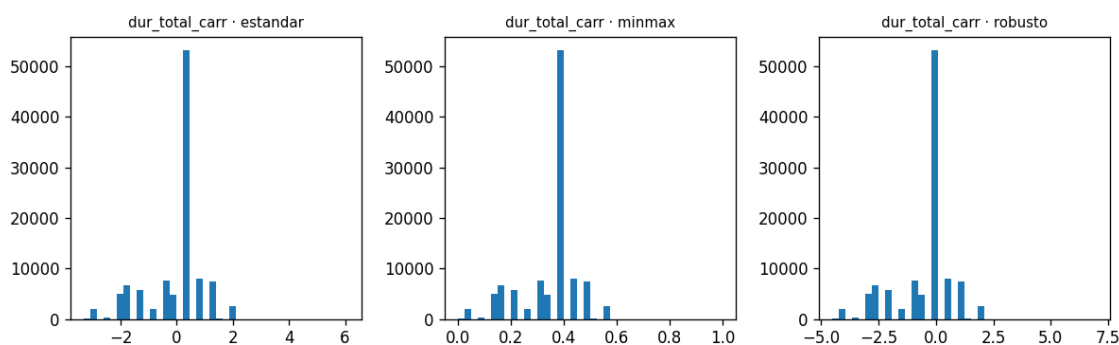


Figura 2. Distribución de `dur_total_carr` bajo los tres escaladores.

Validaciones del dataset resultante y persistencia

`src/validacion.py` ejecuta cinco comprobaciones sobre las 48 columnas de análisis: sin nulos, todas numéricas, cada grupo one-hot suma 1 por fila, sin duplicados y 105.060 filas (105.063 – 3). Además verifica la propiedad del escalador (mediana 0, RIC 1) en las tres columnas escaladas. Las cinco se superaron. El resultado se guardó en `data/processed/titulados_2025_pregrado_univ_limpio.csv` (105.060 × 84, 66,7 MB), se releó y se comprobó su forma; la bitácora exportó 18 decisiones a `docs/bitacora.md`.

5.C Validación técnica y verificación del código

Casos normales, límite y excepciones

Dos archivos de pruebas, ejecutables sin dependencias adicionales (python tests/...) o con pytest: `tests/test_proyecto.py` (8 pruebas, Fase 1) y `tests/test_pipeline.py` (11 pruebas, Fase 2). Resultado: 19/19 superadas (Anexo C).

Tipo	Pruebas	Qué verifican
Caso normal	F1: filtro, generar_subconjunto, generar_readme, verificar_estructura, validador. F2: codigo_a_nulo, imputar_mediana, atipicos_ric, ordinal, one_hot, escaladores	Resultado correcto con la entrada esperada; el DataFrame original no se modifica; la bandera de imputación se crea; el README no tiene marcadores
Caso límite	F1: filtro sin coincidencias, subconjunto existente sin original. F2: columna sin nulos, una sola categoría, sin duplicados, serie vacía	Devuelve vacío o reutiliza; no imputa nada; one-hot de una categoría vale 1; cero eliminaciones; cero atípicos
Excepción	F1: original ausente, columna del filtro ausente, subconjunto sobre el límite, kernel fuera de .venv. F2: columna inexistente, serie no numérica, nulos al escalar, serie constante, DataFrame vacío, min_frec inválido	Se lanza <code>FileNotFoundError</code> , <code>KeyError</code> , <code>ValueError</code> , <code>EnvironmentError</code> o <code>TypeError</code> con mensaje claro

Comprobación de resultados intermedios con evidencia

Cada etapa del notebook F2 imprime sus resultados intermedios (perfil de faltantes, comparación 1900 vs resto, tabla de atípicos, comparación de imputaciones, mapas ordinales, comparación de escaladores, checks) y guarda las figuras en `docs/figuras/`. Las celdas ejecutadas con sus salidas están exportadas en `docs/evidencias/F2_Pipeline_ejecutado.pdf` (Anexo D), y las mismas cifras aparecen en `docs/bitacora.md` y en este informe.

Trazabilidad del proceso de verificación

`docs/verificacion.md` registra qué se verificó, con qué comando o procedimiento, cuándo, con qué resultado y dónde está la evidencia: pruebas automatizadas de F1 y F2, ejecución completa de ambos notebooks

con el kernel del proyecto, comprobación de resultados intermedios contra la bitácora, reproducibilidad del entorno e integridad del repositorio (git status limpio, sin CSV, historial por convención). Cada fila cita el commit vigente (fd71e78) y se actualiza antes de cada commit de entrega.

VI. Notebook F1_Definicion.ipynb (F1)

Ubicación: notebooks/F1/F1_Definicion.ipynb. Refleja exclusivamente la Fase 1: presenta la problemática, el objetivo general y los específicos, y ejecuta las primeras operaciones técnicas del flujo reproducible planificado en el mapa conceptual. La lógica está encapsulada en la clase ProyectoF1 (src/proyecto.py, un dataclass con métodos verificar_entorno, verificar_estructura, aplicar_filtro, generar_subconjunto y generar_readme), probada por tests/test_proyecto.py; el notebook orquesta y documenta con celdas Markdown y comentarios en el código. Se ejecuta completo con el kernel Python (mcdi500) en 8 celdas de código, sin errores (Anexo D).

Sección	Contenido	Salida verificable
1. Contexto, pregunta y objetivos	Situación, problema, pregunta analizable, objetivo general y específicos, alcance; instancia de ProyectoF1 con toda la configuración	Pregunta impresa desde el objeto
2. Entorno reproducible	verificar_entorno(): versión de Python, intérprete y pandas; EnvironmentError si el kernel no es el de .venv	Intérprete dentro de .venv
3. Estructura del repositorio	verificar_estructura() sobre las 9 rutas; assert si falta alguna	OK en las 9 rutas
4. Conjunto de datos	generar_subconjunto(): lee el crudo, filtra, escribe en data/processed/, verifica < 100 MB; validador del curso con assert sobre su código de salida; roles de variables; problemas de calidad	105.063 filas; 55,8 MB; informe en docs/
5. Control de versiones	Reglas de identidad, commits, ramas y nbstripout	—
6. README generado	generar_readme() compone el README; assert de que no quedan marcadores	README.md reescrito
7. Vinculación con el mapa	Tabla implementado / proyectado	—

Correspondencia con este informe: la pregunta, los objetivos, el alcance, la fuente, el filtro y las cifras del capítulo IV son los mismos que imprime el notebook; el README publicado en GitHub es la salida literal de la sección 6. Commits asociados: cfb2d41, 461cdc6, aeeeb4d, 7b83d7c.

VII. Notebooks ejecutables (F2)

Ubicación: notebooks/F2/F2_Pipeline.ipynb (pipeline oficial) y notebooks/F2/F2_borrador_exploracion.ipynb (borrador del equipo, conservado como evidencia de trabajo colaborativo). El pipeline representa exclusivamente la Fase 2: obtención, exploración, limpieza, transformación, escalamiento, validación y persistencia. Se ejecuta completo con el kernel del proyecto (18 celdas de código, contadores ln [1]...[18] en orden), en menos de un minuto, y sus salidas están exportadas en docs/evidencias/F2_Pipeline_ejecutado.pdf (Anexo D). Commits asociados: b9b68b8, 1a7349b, ec3e806, 39b4e47, 4ca6c2e.

Sección del notebook	Funciones de src/ que usa	Evidencia producida
Obtención	—	Forma del subconjunto leído (105.063 × 40) en la bitácora
Paso 6 · Exploración	perfil_faltantes, resumen_categorias, atipicos_ric	Perfil de faltantes y códigos, comparación 1900 vs resto, categorías, tabla de atípicos, Figura 1, medianas por factor

Paso 7 · Limpieza	eliminar_duplicados, eliminar_columnas_constantes, codigo_a_nulo, comparar_imputaciones, imputar_mediana	Conteos, tabla comparativa de imputaciones, cifras de la imputación elegida
Paso 8 · Transformación	codificar_ordinal, derivar_fechas, one_hot, agrupar_raras, binaria_01	Mapas ordinales, descriptivos de edad, conteo de columnas one-hot y categorías agrupadas
Paso 9 · Escalamiento	comparar_escaladores, escalar_robusto	Tabla comparativa y Figura 2
Paso 10 · Validación	validar_dataset_final, verificar_propiedad_escalador; tests/	Cinco checks OK, propiedad del escalador, 11/11 pruebas
Paso 11 · Persistencia	exportar (bitácora)	Ruta, forma y tamaño del archivo procesado; bitácora completa

Trazabilidad: cada decisión aparece con las mismas cifras en la celda que la ejecuta, en docs/bitacora.md y en el capítulo V de este informe, y el historial de GitHub registra quién la incorporó y cuándo.

VIII. Vinculación del mapa conceptual con el avance del proyecto

El mapa conceptual entregado en la Formativa 1 (Figura 3) proyectó el flujo de trabajo reproducible: problema → pregunta → dataset, el ciclo de vida del dato en seis etapas, el ecosistema de herramientas, la estructura del repositorio con commits proyectados y la documentación científica. La tabla indica dónde se materializó cada elemento y qué queda proyectado.

Elemento del mapa	Estado	Dónde se materializó
Problema → pregunta analizable → dataset	Implementado	Notebook F1 secc. 1 y 4; README; informe cap. III–IV
Python + entorno virtual + requirements.txt	Implementado	requirements.txt; verificar_entorno() en F1
Estructura del repositorio, README, .gitignore	Implementado	Repositorio en GitHub; README generado desde ProyectoF1
Git / GitHub, convención de commits	Implementado	19 commits con prefijos docs/feat/fix/test; conflicto resuelto con git pull
Ramas por integrante y pull requests	Proyectado (F3)	Acordado; se aplica desde la Fase 3
Etapas 1–2: definición y obtención con filtro documentado	Implementado	Notebook F1; docs/validacion_subconjunto.md
Etapas 3–4: exploración y transformación → data/processed	Implementado	Notebook F2; src/ titulados_2025_pregrado_univ_limpio.csv
Retorno «obliga a redefinir»	Implementado	La exploración del código 1900 obligó a redefinir su tratamiento (de eliminar a imputar por grupo)
Notebooks: Markdown justifica · código ejecuta · salida evidencia	Implementado	notebooks/F1 y F2; docs/evidencias/
Bitácora de decisiones	Implementado	docs/bitacora.md (18 decisiones)
Funciones reutilizables (src/) y pruebas	Implementado	Clase ProyectoF1 + 6 módulos; 19 pruebas
Referencias APA 7	Implementado	Capítulo IX de este informe
Etapas 5–6: análisis y comunicación; informe final	Proyectado (F3–F4)	—

Mapa conceptual · Flujo de trabajo reproducible del proyecto (Fase 1, proyectado)

Grupo 6 · MCDI500 · Página 2 de 2

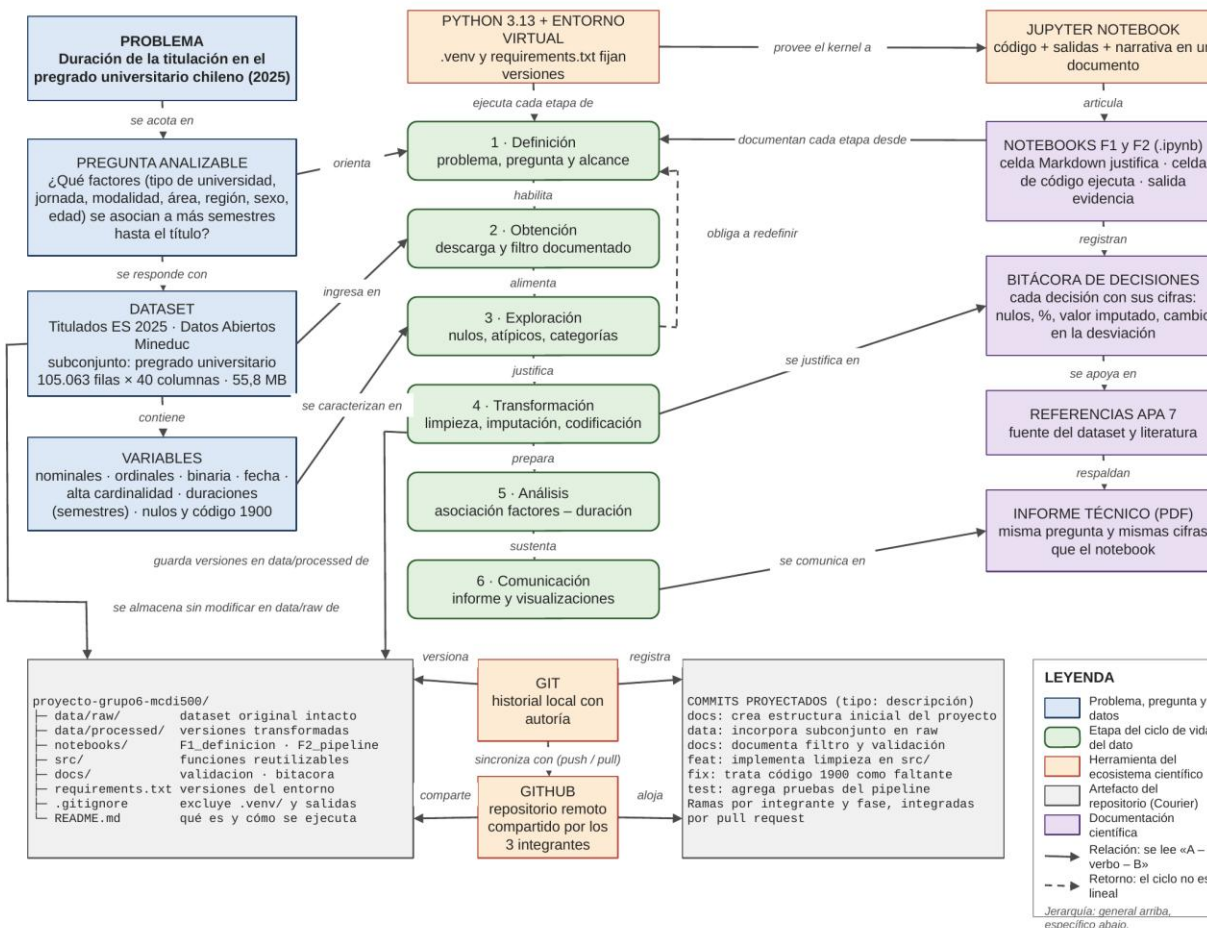


Figura 3. Mapa conceptual del flujo de trabajo reproducible (Formativa 1, Grupo 6).

Coherencia entre planificación y desarrollo: de los trece elementos del mapa, once están materializados en el repositorio, los notebooks, el README o este informe, y dos quedan declarados como proyectados para las fases siguientes. Los elementos que el mapa inicial incluía y que la retroalimentación docente llevó a descartar (regresión lineal, CI/CD, Docker) no aparecen en la versión entregada ni se implementaron.

IX. Bibliografía

Bibliografía docente (material del curso), técnica (documentación oficial) y complementaria (académica), en formato APA 7.^a edición. Todas las fuentes están citadas en el cuerpo del informe.

Bibliografía docente

Salinas Silva, O. (2026). MCDIA500 Programación para la Ciencia de Datos: Semana 1. Fundamentos avanzados de Python y buenas prácticas [Presentación de clase]. Universidad Andrés Bello, UNAB Online.

Salinas Silva, O. (2026). Guía de Git y GitHub del curso MCDIA500 (versiones Windows y macOS) [Material de curso]. Universidad Andrés Bello, UNAB Online.

Bibliografía técnica (documentación oficial)

The pandas development team. (2025). pandas documentation (versión 2.3). <https://pandas.pydata.org/docs/>

NumPy Developers. (2025). NumPy documentation (versión 2.0). <https://numpy.org/doc/stable/>

Project Jupyter. (2025). JupyterLab documentation (versión 4.x). <https://jupyterlab.readthedocs.io/>

Git Project. (2025). Git reference documentation. <https://git-scm.com/docs>

Bibliografía complementaria (académica)

The Turing Way Community. (2022). The Turing Way: A handbook for reproducible, ethical and collaborative research (versión 1.0.2). Zenodo. <https://doi.org/10.5281/zenodo.3233853>

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology*, 13(6), e1005510. <https://doi.org/10.1371/journal.pcbi.1005510>

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., ... Willing, C. (2016). Jupyter Notebooks: A publishing format for reproducible computational workflows. En F. Loizides & B. Schmidt (Eds.), *Positioning and Power in Academic Publishing* (pp. 87–90). IOS Press. <https://doi.org/10.3233/978-1-61499-649-1-87>

McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>

Fuente de datos

Ministerio de Educación de Chile. (2026). Titulados de educación superior 2025 [Conjunto de datos]. Datos Abiertos Mineduc, Centro de Estudios. <https://datosabiertos.mineduc.cl/titulados-en-educacion-superior/>

Anexos técnicos

Anexo A · Bitácora de decisiones

Contenido íntegro de docs/bitacora.md, generado por el notebook F2 (Fase 2):

```
[F2 · obtención] lectura de data/processed/titulados_2025_pregrado_univ.csv → 105,063 filas × 40
columnas (sep ';', UTF-8)
[F2 · exploración] código 1900 en anio_ing_carr_ori: 13,396 registros (12,8 %) con duración
mediana 6 vs 10 semestres → no es un faltante aleatorio
[F2 · exploración] dur_total_carr: mediana 10 semestres, RIC 2, 9,845 atípicos (9.4 %) por regla
1,5·RIC → hay valores extremos
[F2 · limpieza] duplicados exactos: 3 filas eliminadas (0.003 %) → 105,060 filas
[F2 · limpieza] columnas constantes eliminadas (un solo valor tras el filtro de F1):
['cat_periodo', 'tipo_inst_1', 'nivel_global']
[F2 · limpieza] anio_ing_carr_ori: código 1900 tratado como faltante en 13,396 filas (12.8 %);
sem_ing_carr_ori: código 0 tratado como faltante en 13,396 filas (12.8 %)
[F2 · limpieza] anio_ing_carr_ori: 13,396 nulos (12.75 %) imputados con mediana por
anio_ing_carr_act; cambio en la desviación estándar +2.82 %; bandera anio_ing_carr_ori_imputada
[F2 · limpieza] sem_ing_carr_ori: 13,396 nulos (12.75 %) imputados con la mediana = 1
[F2 · limpieza] nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se conservan sin imputar: son
etiquetas de texto, no variables de análisis; mrun con 7 nulos se conserva porque el análisis no
cuenta personas sino títulos
[F2 · transformación] ordinales codificadas con orden declarado: rango_edad (6 niveles) y
nivel_carrera_1 (4 niveles); 0 registros con rango_edad [] quedan como faltante
[F2 · transformación] fechas: fecha_obtencion_titulo y fec_nac_alu parseadas; derivadas
mes_titulacion y edad_titulacion (mediana 25.6 años); 0 'Sin Información' de rango_edad
recuperados desde edad_titulacion
[F2 · transformación] one-hot encoding de 5 nominales → 36 columnas 0/1
[F2 · transformación] nomb_carrera: 968 categorías con < 100 registros agrupadas en 'OTRA' → 129
categorías
[F2 · transformación] gen_alu recodificada como mujer (0/1): 56.3% mujeres
[F2 · escalamiento] RobustScaler (mediana 0, RIC 1) aplicado a ['dur_total_carr',
'dur_estudio_carr', 'edad_titulacion']: coherente con la imputación por mediana y con los
atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho por el máximo
de 24 semestres
[F2 · validación] 5 comprobaciones superadas sobre 48 columnas de análisis; propiedad mediana 0 /
RIC 1 verificada
[F2 · validación] pruebas unitarias de src/ (tests/test_pipeline.py): 11/11 pruebas superadas
[F2 · persistencia] titulados_2025_pregrado_univ_limpio.csv: 105,060 × 84, releído y verificado
(forma y columnas)
```

Anexo B · Columnas del dataset procesado

data/processed/titulados_2025_pregrado_univ_limpio.csv: 105.060 filas × 84 columnas. Originales conservadas (37): las 40 del subconjunto menos las 3 constantes. Derivadas (47): anio_ing_carr_ori_imputada, sem_ing_carr_ori_imputada, rango_edad_ord, nivel_carrera_1_ord, mes_titulacion, edad_titulacion, 36 columnas one-hot (tipo_inst_2_*, jornada_*, modalidad_*, area_conocimiento_*, region_sede_*), nomb_carrera_agrupada, mujer, y las escaladas dur_total_carr_esc, dur_estudio_carr_esc, edad_titulacion_esc.

Anexo C · Salida de las pruebas automatizadas

```
$ python tests/test_proyecto.py
OK test_excepciones
OK test_filtro_conserva_solo_pregrado_universitario
OK test_filtro_sin_coincidencias_devuelve_vacio
OK test_generar_subconjunto_escribe_archivo
OK test_readme_contiene_pregunta_e_integrantes
OK test_subconjunto_existente_se_reutiliza_sin_original
OK test_validador_detecta_separador_y_alertas
OK test_verificar_estructura_detecta_faltantes
```

8/8 pruebas superadas

```
$ python tests/test_pipeline.py
```

```
OK test_atipicos_ric_detecta_extremo
OK test_atipicos_serie_vacia
OK test_codigo_a_nulo_normal
OK test_columna_sin_nulos_no_imputa_nada
OK test_eliminar_duplicados_sin_duplicados
OK test_escaladores_cumplen_propiedad
OK test_excepciones
OK test_imputar_mediana_crea_bandera
OK test_one_hot_suma_uno
OK test_ordinal_respeto_orden_declarado
OK test_una_sola_categoria_one_hot
```

11/11 pruebas superadas

Anexo D · Notebooks ejecutados

Se adjuntan a continuación, como parte de este PDF, docs/evidencias/F1_Definicion_ejecutado.pdf y docs/evidencias/F2_Pipeline_ejecutado.pdf, con todas las celdas ejecutadas en orden y sus salidas.

Fase 1 · Definición del problema, entorno reproducible y estructura del proyecto

MCDIA500 · Programación para la Ciencia de Datos · Grupo 6 Integrantes: *Fernanda Ovalle Román · Sebastián Cajales Cid · César Lorca Bacián* Repositorio: <https://github.com/Feroroman/proyecto-grupo6-mcdi500>

Este notebook documenta la Fase 1 del proyecto transversal. No analiza datos: deja las condiciones para que el análisis de la Fase 2 sea reproducible, trazable y compartido. La lógica técnica vive en la clase `ProyectoF1` (`src/proyecto.py`), probada en `tests/test_proyecto.py`; aquí se orquesta y se documenta.

Antes de entregar: **Kernel** → **Restart Kernel and Run All Cells**. Se ejecuta con el kernel *Python (mcdi500)*; las rutas se resuelven desde la raíz del repositorio.

1. Contexto problematizador y pregunta analizable

Situación. En Chile la duración real de las carreras universitarias suele superar la duración formal, con costos para estudiantes e instituciones. El Mineduc publica cada año los registros de titulados, pero no está claro qué características de la carrera y de la institución acompañan una titulación más larga.

Problema. No se conoce qué factores observables se asocian a una mayor duración hasta el título.

Pregunta analizable. ¿Qué factores observables —tipo de universidad (CRUCH/privada), jornada, modalidad, área de conocimiento, región de la sede, sexo y rango etario— se asocian a un mayor número de semestres hasta la obtención del título (`dur_total_carr`) entre los titulados de pregrado universitario de 2025?

Objetivo general. Caracterizar la duración de la titulación en el pregrado universitario chileno (cohorte 2025) e identificar los factores observables asociados a una mayor duración, mediante un flujo de trabajo reproducible en Python.

Objetivos específicos. 1. Obtener y verificar el conjunto de datos de titulados 2025 del Mineduc, documentando origen, variables y problemas de calidad (F1). 2. Construir un pipeline de limpieza y transformación que justifique cada decisión con cifras (F2). 3. Implementar funciones y clases reutilizables para calcular y comparar la duración por grupos (F3). 4. Comunicar los hallazgos con visualizaciones e informe técnico (F4).

Alcance. Solo pregrado en universidades (quedan fuera IP, CFT, postítulos y posgrados). Análisis descriptivo y de asociación; no se construyen modelos predictivos.

Por qué es analizable. Los datos existen (`dur_total_carr` como variable de respuesta y los factores como columnas del mismo registro), los conceptos se miden con esas variables y el alcance está acotado.

In [1]:

```
# Ubicar la raíz del repositorio (funciona desde notebooks/F1/ o desde la raíz) y
cargar el módulo del proyecto

from pathlib import Path

import os, sys

RAIZ = Path.cwd()

while not (RAIZ / "README.md").exists() and RAIZ.parent != RAIZ:
```

```

RAIZ = RAIZ.parent
os.chdir(RAIZ); sys.path.insert(0, str(RAIZ))

from src.proyecto import ProyectoF1 # clase que encapsula configuración y
operaciones de la Fase 1

# Toda la configuración del proyecto en un solo objeto: el README, las rutas y el
filtro se derivan de aquí
proyecto = ProyectoF1(
    grupo="Grupo 6",
    integrantes=["Fernanda Ovalle Román", "Sebastián Cajales Cid", "César Lorca
Bacián"],
    repositorio="https://github.com/Feroroman/proyecto-grupo6-mcdi500",
    pregunta=("¿Qué factores observables (tipo de universidad, jornada, modalidad,
área de conocimiento, "
        "región de la sede, sexo y rango etario) se asocian a un mayor número
de semestres hasta la "
        "obtención del título entre los titulados de pregrado universitario de
2025?"),
    fuente_nombre="Titulados de Educación Superior 2025 - Datos Abiertos Mineduc",
    fuente_url="https://datosabiertos.mineduc.cl/",
    fuente_licencia=("Datos públicos del Estado de Chile, de acceso libre y
gratuito, publicados por el Centro de Estudios "
        "del Mineduc; el portal no declara una licencia Creative
Commons específica y se reutilizan con "
        "atribución a la fuente conforme a la Ley 20.285 sobre acceso a
la información pública"),
    archivo_original=Path("data/raw/20260817_Titulados_Ed_Superior_2025_WEB.csv"),
    archivo_subconjunto=Path("data/processed/titulados_2025_pregrado_univ.csv"),
    filtro={"nivel_global": "Pregrado", "tipo_inst_1": "Universidades"},
    variable_respuesta="dur_total_carr",
    factores=["tipo_inst_2", "jornada", "modalidad", "area_conocimiento",
"region_sede", "gen_alu", "rango_edad"],
)
print("Raíz del proyecto:", RAIZ)
print("Pregunta:", proyecto.pregunta)

```

Out:

```

Raíz del proyecto:
/tmp/claude-0/-home-claude/7d5d987b-d69f-5ff6-b6ac-07de88f524e4/scratchpad/repo

Pregunta: ¿Qué factores observables (tipo de universidad, jornada, modalidad, área de
conocimiento, región de la sede, sexo y rango etario) se asocian a un mayor número de
semestres hasta la obtención del título entre los titulados de pregrado universitario
de 2025?

```

2. Entorno reproducible

Reproducibilidad = mismos datos + mismo código + mismo entorno. El entorno virtual `.venv` aísla las librerías, `requirements.txt` declara sus versiones y el kernel del notebook debe ser el del proyecto (**Python (mcdi500)**). `verificar_entorno()` lanza un error explícito si no es así.

In [2]:

```

info = proyecto.verificar_entorno(exigir_venv=True) # falla si el kernel no es el
de .venv

for k, v in info.items():

```

```
| print(f"{k:<11}: {v}")
```

| Out:

```
| python      : 3.11.15
| interprete  : /tmp/claude-0/.venv/bin/python3
| pandas     : 3.0.2
```

3. Estructura del repositorio

Cada carpeta corresponde a una etapa del ciclo de vida del dato. El dato crudo (data/raw/) nunca se modifica; cada transformación produce una versión nueva en data/processed/.

| In [3]:

```
| estado = proyecto.verificar_estructura(RAIZ)
| for ruta, existe in estado.items():
|     print(f"'OK ' if existe else 'FALTA' {ruta:<18}
|     {proyecto.estructura[ruta]}")
| assert all(estado.values()), "Faltan elementos de la estructura del repositorio"
```

| Out:

```
| OK    data/raw/          dato crudo descargado del portal; no se modifica ni se
| versiona
| OK    data/processed/    derivados regenerables: subconjunto (F1) y versiones limpias
| (F2)
| OK    notebooks/        F1/F1_Definicion.ipynb · F2/F2_Pipeline.ipynb
| OK    src/              módulos: proyecto, exploracion, limpieza, transformacion,
| escalado, validacion, bitacora
| OK    tests/            pruebas de las funciones (caso normal, límite, excepción)
| OK    docs/             validación del dataset, bitácora, verificación, figuras,
| evidencias
| OK    requirements.txt   versiones declaradas del entorno
| OK    .gitignore        qué NO se versiona: .venv/, todos los CSV, salidas de
| notebooks, .idea/
| OK    README.md         qué es el proyecto y cómo ejecutarlo desde cero
```

4. Conjunto de datos: origen, verificación y subconjunto

La base nacional completa (175,6 MB) es el **dato crudo**: se descarga del portal, se deja en data/raw/ y no se modifica ni se versiona (supera el límite de 100 MB de GitHub). El proyecto trabaja sobre un **subconjunto derivado**: titulados de **pregrado en universidades**. El filtro es una decisión técnica: mantiene una población homogénea para la pregunta (la duración de un magíster o de una carrera técnica no es comparable con la de un pregrado universitario).

El subconjunto es un derivado: se escribe en data/processed/ y tampoco se versiona; cualquier integrante lo regenera con generar_subconjunto(), que además verifica que quede bajo 100 MB.

| In [4]:

```
| import pandas as pd
| df, mensaje = proyecto.generar_subconjunto(limite_mb=100)
| print(mensaje)
| print(f"Tamaño en disco: {proyecto.archivo_subconjunto.stat().st_size /
| 1_048_576:.1f} MB (límite GitHub: 100 MB)")
```

| Out:

```
| Base completa: 328,998 filas → subconjunto: 105,063 filas guardado en
| data/processed/titulados_2025_pregrado_univ.csv
```

Tamaño en disco: 55.8 MB (límite GitHub: 100 MB)

In [5]:

```
# Verificación con el validador del curso (src/validar_dataset.py) → docs/validacion_subconjunto.md

import subprocess

res = subprocess.run([sys.executable, "src/validar_dataset.py",
str(proyecto.archivo_subconjunto), "--sep", proyecto.separador,
"--informe", "docs/validacion_subconjunto.md"],
capture_output=True, text=True)

assert res.returncode == 0, res.stderr # si el validador falla, el notebook debe fallar

print(res.stdout)

print(Path("docs/validacion_subconjunto.md").read_text(encoding="utf-8").split("##
Alertas")[-1])
```

Out:

```
Informe guardado en docs/validacion_subconjunto.md
Alertas: 4
```

- ⚠ Hay 3 filas duplicadas.
- ⚠ `cat\_periodo` es constante (un solo valor).
- ⚠ `tipo\_inst\_1` es constante (un solo valor).
- ⚠ `nivel\_global` es constante (un solo valor).

Roles analíticos de las variables

Cada rol exige un tratamiento distinto en la Fase 2. Declararlos aquí es lo que después justifica cada decisión.

In [6]:

```
ROLES = {
    "identificador": ["mrun"],
    "numerica (semestres)": ["dur_estudio_carr", "dur_proceso_tit",
"dur_total_carr"],
    "numerica discreta": ["anio_ing_carr_ori", "anio_ing_carr_act", "version"],
    "binaria": ["gen_alu"],
    "nominal": ["tipo_inst_2", "jornada", "modalidad",
"area_conocimiento", "region_sede", "tipo_plan_carr"],
    "ordinal": ["rango_edad", "nivel_carrera_1"],
    "fecha": ["fecha_obtencion_titulo", "fec_nac_alu"],
    "alta_cardinalidad": ["nomb_carrera", "nomb_inst", "nomb_sede",
"comuna_sede"],
    "constante_descartar": ["cat_periodo", "tipo_inst_1", "nivel_global"],
}

faltantes = df.isna().mean().mul(100).round(1)

for rol, cols in ROLES.items():
    print(f"\n{rol}:")
    for c in cols:
        print(f" {c:<24} nulos {faltantes[c]:>5}% únicos {df[c].nunique():>7,}")
```

Out:

identificador:					
	mrún	nulos	0.0%	únicos	104,595
numerica (semestres):					
	dur_estudio_carr	nulos	0.0%	únicos	15
	dur_proceso_tit	nulos	0.0%	únicos	7
	dur_total_carr	nulos	0.0%	únicos	16
numerica_discreta:					
	anio_ing_carr_ori	nulos	0.0%	únicos	45
	anio_ing_carr_act	nulos	0.0%	únicos	42
	version	nulos	0.3%	únicos	8
binaria:					
	gen_alu	nulos	0.0%	únicos	2
nominal:					
	tipo_inst_2	nulos	0.0%	únicos	2
	jornada	nulos	0.0%	únicos	5
	modalidad	nulos	0.0%	únicos	3
	area_conocimiento	nulos	0.0%	únicos	10
	region_sede	nulos	0.0%	únicos	16
	tipo_plan_carr	nulos	0.0%	únicos	3
ordinal:					
	rango_edad	nulos	0.0%	únicos	6
	nivel_carrera_1	nulos	0.0%	únicos	4
fecha:					
	fecha_obtencion_titulo	nulos	0.0%	únicos	355
	fec_nac_alu	nulos	0.0%	únicos	585
alta_cardinalidad:					
	nomb_carrera	nulos	0.0%	únicos	1,096
	nomb_inst	nulos	0.0%	únicos	55
	nomb_sede	nulos	0.0%	únicos	109
	comuna_sede	nulos	0.0%	únicos	56
constante_descartar:					
	cat_periodo	nulos	0.0%	únicos	1
	tipo_inst_1	nulos	0.0%	únicos	1
	nivel_global	nulos	0.0%	únicos	1

Problemas de calidad detectados (insumo de la Fase 2)

Se miden aquí; se resuelven y justifican en notebooks/F2/F2_Pipeline.ipynb.

In [7]:

```
cod_1900 = int((df["anio_ing_carr_ori"] == 1900).sum())
print(f"anio_ing_carr_ori = 1900 (código «sin información»): {cod_1900:,} filas
      ({100*cod_1900/len(df):.1f} %)")
print(f"Filas duplicadas: {df.duplicated().sum():,}")
print(f"mr_run nulo: {df['mr_run'].isna().sum():,}")
print(f"nombre_titulo nulo: {100*df['nombre_titulo'].isna().mean():.1f} % .
      nombre_grado nulo: {100*df['nombre_grado'].isna().mean():.1f} %")
print(f"Columnas constantes: {[c for c in df.columns if df[c].nunique() == 1]}")
print("\nDistribución de la variable de respuesta (semestres):")
print(df[proyecto.variable_respuesta].describe().round(2).to_string())
```

Out:

```
anio_ing_carr_ori = 1900 (código «sin información»): 13,396 filas (12.8 %)
Filas duplicadas: 3
mr_run nulo: 7
nombre_titulo nulo: 2.9 % . nombre_grado nulo: 8.0 %
Columnas constantes: ['cat_periodo', 'tipo_inst_1', 'nivel_global']

Distribución de la variable de respuesta (semestres):
count    105063.00
mean         9.07
std         2.44
min         1.00
25%         8.00
50%        10.00
75%        10.00
max         24.00
```

5. Control de versiones y trabajo colaborativo

- **Git** registra el historial local con autoría; **GitHub** aloja el repositorio remoto compartido por los tres integrantes.
- Convención de commits tipo: descripción con los prefijos docs, data, feat, fix, test.
- Una rama por integrante y fase, integrada a main mediante pull request; conflictos resueltos con git pull, nunca con --force.
- nbstripout activo en los tres equipos: limpia las salidas del notebook antes de cada commit.
- Ningún CSV se versiona (ver .gitignore): el repositorio contiene el código que reproduce los datos, no los datos.

6. README generado desde la configuración

El README se compone desde el objeto proyecto, así la documentación no se desactualiza respecto del código.

In [8]:

```
readme = proyecto.generar_readme()
Path("README.md").write_text(readme, encoding="utf-8")
assert "TU_USUARIO" not in readme and "completar" not in readme.lower() # sin
marcadores pendientes
```

```

| print(readme[:1200], "\n[...]")
Out:

| # Duración de la titulación en el pregrado universitario chileno (2025)

| **MCDI500 · Programación para la Ciencia de Datos** · Grupo 6

| ## Integrantes
| - Fernanda Ovalle Román
| - Sebastián Cajales Cid
| - César Lorca Bacián

| ## Problemática y pregunta
| La duración real de las carreras universitarias en Chile suele superar la formal. Este
| proyecto analiza qué
| factores observables acompañan una titulación más larga.

| **Pregunta analizable:** ¿Qué factores observables (tipo de universidad, jornada,
| modalidad, área de conocimiento, región de la sede, sexo y rango etario) se asocian a
| un mayor número de semestres hasta la obtención del título entre los titulados de
| pregrado universitario de 2025?

| **Alcance:** solo pregrado en universidades; análisis descriptivo y de asociación, sin
| modelos predictivos.

| ## Conjunto de datos
| - Fuente: Titulados de Educación Superior 2025 – Datos Abiertos Mineduc –
| https://datosabiertos.mineduc.cl/
| - Licencia: Datos públicos del Estado de Chile, de acceso libre y gratuito, publicados
| por el Centro de Estudios del Mineduc; el portal no declara una licencia Creative
| Commons específica y se reutilizan con atribución a la fuente conforme a la Ley 20.285
| sobre acc
| [...]

```

7. Vinculación con el mapa conceptual (Formativa 1)

Elemento del mapa	Estado
Problema → pregunta analizable → dataset	Implementado (secciones 1 y 4)
Python + entorno virtual + requirements.txt	Implementado (sección 2, verificar_entorno)
Estructura del repositorio, README, .gitignore	Implementado (secciones 3 y 6)
Git / GitHub, convención de commits	Implementado (sección 5; historial en GitHub)
Funciones y clases reutilizables (src/) con pruebas	Implementado: ProyectoF1 + tests/test_proyecto.py
Ciclo de vida: exploración → transformación → data/processed	Implementado en

Elemento del mapa	Estado
	<code>notebooks/F2/F2_Pipeline.ipynb</code>
Bitácora de decisiones	Implementado en la Fase 2 (<code>docs/bitacora.md</code>)
Análisis y comunicación, informe final	Proyectado: Fases 3 y 4

Fase 2 · Pipeline de obtención, exploración, limpieza, transformación y validación

MCDIA500 · Programación para la Ciencia de Datos · Grupo 6 Integrantes: *Fernanda Ovalle Román · Sebastián Cajales Cid · César Lorca Bacián*

Este notebook implementa el pipeline de la Fase 2 sobre el subconjunto definido en la Fase 1 (titulados de pregrado universitario 2025). Regla de trabajo: **medir primero, decidir después y justificar con la medición**. Cada decisión queda registrada en la bitácora (docs/bitacora.md) con sus cifras, y las funciones que la ejecutan viven en src/ con pruebas en tests/.

Antes de entregar: **Kernel → Restart Kernel and Run All Cells**. Requiere que data/processed/titulados_2025_pregrado_univ.csv exista (lo genera notebooks/F1/F1_Definicion.ipynb).

In [1]:

```
from pathlib import Path
import os, sys
RAIZ = Path.cwd()
while not (RAIZ / "README.md").exists() and RAIZ.parent != RAIZ:
    RAIZ = RAIZ.parent
os.chdir(RAIZ); sys.path.insert(0, str(RAIZ))
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from src import exploracion, limpieza, transformacion, escalado, validacion
from src.bitacora import registrar, exportar

FIG = RAIZ / "docs" / "figuras"; FIG.mkdir(parents=True, exist_ok=True)
def guardar_figura(nombre):
    ruta = FIG / f"{nombre}.png"; plt.tight_layout(); plt.savefig(ruta, dpi=120);
    plt.close(); print("figura guardada:", ruta.relative_to(RAIZ))

pd.set_option("display.width", 160); pd.set_option("display.max_columns", 30)
print("Raíz:", RAIZ, "· Python", sys.version.split()[0], "· pandas", pd.__version__)
```

Out:

```
Raíz: /tmp/claude-0/-home-claude/7d5d987b-d69f-5ff6-b6ac-07de88f524e4/scratchpad/repo ·
Python 3.11.15 · pandas 3.0.2
```

Obtención

El dato crudo (data/raw/) no se toca. Se parte del subconjunto derivado que produce el notebook F1 y se verifica su forma antes de hacer nada.

In [2]:

```
ENTRADA = Path("data/processed/titulados_2025_pregrado_univ.csv")
assert ENTRADA.exists(), "Ejecuta primero notebooks/F1/F1_Definicion.ipynb para
```

```

| generar el subconjunto"
| df0 = pd.read_csv(ENTRADA, sep=";", encoding="utf-8", low_memory=False)
| N0 = len(df0)
| print(f"Subconjunto: {df0.shape[0]:,} filas × {df0.shape[1]} columnas")
| registrar("F2 · obtención", f"lectura de {ENTRADA} → {df0.shape[0]:,} filas ×
| {df0.shape[1]} columnas (sep ';', UTF-8)")

```

Out:

```

| Subconjunto: 105,063 filas × 40 columnas
| [F2 · obtención] lectura de data/processed/titulados_2025_pregrado_univ.csv → 105,063
| filas × 40 columnas (sep ';', UTF-8)

```

Paso 6 · Exploración: medir antes de tocar nada

Cuatro mediciones: dimensiones y tipos, valores faltantes (incluidos los códigos de relleno), categorías observadas y valores atípicos.

In [3]:

```

| # 1) Dimensiones y tipos
| print(df0.dtypes.value_counts().rename("columnas").to_string())
| print()
| # 2) Faltantes reales + códigos de relleno (1900 en año de ingreso, 0 en semestre de
| ingreso)
| perfil = exploracion.perfil_faltantes(df0, codigos={"anio_ing_carr_ori": 1900,
| "sem_ing_carr_ori": 0, "cod_sede": 0})
| print(perfil[perfil["pct_total"] > 0].to_string())

```

Out:

```

| str      24
| int64    13
| float64   3

```

	nulos	pct_nulos	codigo_relleno	pct_codigo	pct_total
columna					
sem_ing_carr_ori	0	0.00	13396	12.75	12.75
anio_ing_carr_ori	0	0.00	13396	12.75	12.75
nombre_grado	8437	8.03	0	0.00	8.03
nombre_titulo	2997	2.85	0	0.00	2.85
cod_carrera	300	0.29	0	0.00	0.29
version	290	0.28	0	0.00	0.28
mrnun	7	0.01	0	0.00	0.01

In [4]:

```

| # ¿Los registros con código 1900 tienen duraciones distintas? (decide si el código
| es informativo o solo faltante)
| m1900 = df0["anio_ing_carr_ori"] == 1900
| comp = pd.DataFrame({"con código 1900": df0.loc[m1900, ["dur_estudio_carr",
| "dur_total_carr"]].describe().loc[["count", "mean", "50%", "max"]].round(2).stack(),
| "sin código": df0.loc[~m1900, ["dur_estudio_carr",
| "dur_total_carr"]].describe().loc[["count", "mean", "50%",
| "max"]].round(2).stack()})
| print(comp.to_string())
| print()
| print("anio_ing_carr_ori == anio_ing_carr_act en registros válidos:",

```

```
f"{df0.loc[~m1900, 'anio_ing_carr_ori'] == df0.loc[~m1900, 'anio_ing_carr_act']).mean():.1%}")
```

Out:

		con código 1900	sin código
count	dur_estudio_carr	13396.00	91667.00
	dur_total_carr	13396.00	91667.00
mean	dur_estudio_carr	5.55	9.54
	dur_total_carr	5.55	9.59
50%	dur_estudio_carr	6.00	10.00
	dur_total_carr	6.00	10.00
max	dur_estudio_carr	14.00	24.00
	dur_total_carr	14.00	24.00

anio_ing_carr_ori == anio_ing_carr_act en registros válidos: 91.2%

In [5]:

```
# 3) Categorías observadas: detecta variantes de escritura y niveles raros
cat = exploracion.resumen_categorias(df0, ["tipo_inst_2", "jornada", "modalidad",
"rango_edad", "nivel_carrera_1", "tipo_plan_carr", "area_conocimiento"], top=6)
for c, r in cat.iterrows():
    print(f"{c} ({r['n_categorias']}): {r['mas_frecuentes']}")
print()
print("nomb_carrera:", df0["nomb_carrera"].nunique(), "valores;",
(df0["nomb_carrera"].value_counts() < 100).sum(), "con menos de 100 registros")
print("nomb_inst : ", df0["nomb_inst"].nunique(), "valores;",
(df0["nomb_inst"].value_counts() < 100).sum(), "con menos de 100 registros")
```

Out:

```
tipo_inst_2 (2): Universidades CRUCH (55,572), Universidades Privadas (49,491)
jornada (5): Diurna (82,197), Vespertina (12,431), A Distancia (9,064), Semipresencial
(828), Otra (543)
modalidad (3): Presencial (94,874), No Presencial (9,064), Semipresencial (1,125)
rango_edad (6): 20 a 24 Años (46,399), 25 a 29 Años (36,078), 30 a 34 Años (9,798), 40
y más años (6,868), 35 a 39 Años (5,759), 15 a 19 Años (161)
nivel_carrera_1 (4): Profesional Con Licenciatura (93,629), Técnico de Nivel Superior
(5,329), Profesional Sin Licenciatura (3,108), Licenciatura No Conducente a Título
(2,997)
tipo_plan_carr (3): Plan Regular (86,937), Plan Regular de Continuidad (15,472), Plan
Especial (2,654)
area_conocimiento (10): Tecnología (23,705), Salud (22,506), Ciencias Sociales
(16,371), Administración y Comercio (14,295), Educación (11,833), Derecho (5,074)

nomb_carrera: 1096 valores; 968 con menos de 100 registros
nomb_inst : 55 valores; 3 con menos de 100 registros
```

In [6]:

```
# 4) Valores atípicos por rango intercuartílico
at = pd.DataFrame({c: exploracion.atipicos_ric(df0[c]) for c in ["dur_estudio_carr",
"dur_proceso_tit", "dur_total_carr"]}).T
print(at.to_string())
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2))
for ax, c in zip(axes, ["dur_estudio_carr", "dur_proceso_tit", "dur_total_carr"]):
    ax.boxplot(df0[c].dropna(), vert=True); ax.set_title(c, fontsize=9);
    ax.set_ylabel("semestres", fontsize=8)
```



```

guardar_figura("f2_01_atipicos_duraciones")

registrar("F2 · exploración", f"código 1900 en anio_ing_carr_ori:
{int(m1900.sum()):,} registros (12,8 %) con duración mediana "

        f"{df0.loc[m1900,'dur_total_carr'].median():.0f} vs
{df0.loc[~m1900,'dur_total_carr'].median():.0f} semestres → no es un faltante
aleatorio")

registrar("F2 · exploración", f"dur_total_carr: mediana
{df0.dur_total_carr.median():.0f} semestres, RIC
{at.loc['dur_total_carr','ric']:.0f}, "

        f"{int(at.loc['dur_total_carr','n_atipicos']):,} atípicos
({at.loc['dur_total_carr','pct']:.1f} %) por regla 1,5·RIC → hay valores extremos")

```

Out:

```

      q1    q3   ric  lim_inf  lim_sup  n_atipicos   pct
dur_estudio_carr  8.0  10.0  2.0     5.0    13.0    10162.0  9.67
dur_proceso_tit   0.0   1.0  1.0    -1.5     2.5     312.0  0.30
dur_total_carr    8.0  10.0  2.0     5.0    13.0     9845.0  9.37

figura guardada: docs/figuras/f2_01_atipicos_duraciones.png

[F2 · exploración] código 1900 en anio_ing_carr_ori: 13,396 registros (12,8 %) con
duración mediana 6 vs 10 semestres → no es un faltante aleatorio

[F2 · exploración] dur_total_carr: mediana 10 semestres, RIC 2, 9,845 atipicos (9.4 %)
por regla 1,5·RIC → hay valores extremos

```

In [7]:

```

# Vista previa de la pregunta: mediana de duración por factor (solo descriptivo; el
análisis es Fase 3-4)

for f in ["tipo_inst_2", "jornada", "modalidad"]:

    print(df0.groupby(f)["dur_total_carr"].agg(n="size", mediana="median",
media="mean").round(2).sort_values("mediana").to_string()); print()

```

Out:

```

      n  mediana  media
tipo_inst_2
Universidades CRUCH    55572    10.0   9.50
Universidades Privadas  49491    10.0   8.59

```

```

      n  mediana  media
jornada
A Distancia    9064     6.0   5.37
Vespertina    12431     6.0   6.41
Semipresencial  828     7.0   6.70
Otra           543     8.0   6.97
Diurna        82197    10.0   9.92

```

```

      n  mediana  media
modalidad
No Presencial  9064     6.0   5.37
Semipresencial 1125     6.0   6.68
Presencial    94874    10.0   9.46

```

Paso 7 · Limpieza e imputación: comparar antes de decidir

Decisiones, en orden: duplicados → columnas constantes → códigos de relleno como faltantes

→ estrategia de imputación elegida por su efecto sobre la dispersión.

In [8]:

```
df1, n_dup = limpieza.eliminar_duplicados(df0)

registrar("F2 · limpieza", f"duplicados exactos: {n_dup} filas eliminadas
({100*n_dup/N0:.3f} %) → {len(df1):,} filas")

df1, constantes = limpieza.eliminar_columnas_constantes(df1)

registrar("F2 · limpieza", f"columnas constantes eliminadas (un solo valor tras el
filtro de F1): {constantes}")

df1, n1900 = limpieza.codigo_a_nulo(df1, "anio_ing_carr_ori", 1900)
df1, n0 = limpieza.codigo_a_nulo(df1, "sem_ing_carr_ori", 0)

registrar("F2 · limpieza", f"anio_ing_carr_ori: código 1900 tratado como faltante en
{n1900:,} filas ({100*n1900/len(df1):.1f} %); "
        f"sem_ing_carr_ori: código 0 tratado como faltante en {n0:,} filas
({100*n0/len(df1):.1f} %)")
```

Out:

```
[F2 · limpieza] duplicados exactos: 3 filas eliminadas (0.003 %) → 105,060 filas
[F2 · limpieza] columnas constantes eliminadas (un solo valor tras el filtro de F1):
['cat_periodo', 'tipo_inst_1', 'nivel_global']
[F2 · limpieza] anio_ing_carr_ori: código 1900 tratado como faltante en 13,396 filas
(12.8 %); sem_ing_carr_ori: código 0 tratado como faltante en 13,396 filas (12.8 %)
```

In [9]:

```
# Comparación de estrategias para anio_ing_carr_ori: eliminar, media, mediana,
mediana por grupo (anio_ing_carr_act)

comparacion = limpieza.comparar_imputaciones(df1, "anio_ing_carr_ori",
grupo="anio_ing_carr_act")

print(comparacion.to_string())
```

Out:

	n	media	desv	cambio_desv_pct
estrategia				
observados (referencia)	91664	2019.449	2.561	0.000
eliminar filas	91664	2019.449	2.561	0.000
imputar media	105060	2019.449	2.392	-6.593
imputar mediana	105060	2019.519	2.399	-6.318
imputar mediana por anio_ing_carr_act	105060	2019.812	2.633	2.821

Lectura de la tabla. Los registros con código 1900 **no son aleatorios**: su duración mediana es de 6 semestres frente a 10 en el resto (la exploración lo mostró), lo que sugiere estudiantes que cambiaron de carrera y cuyo ingreso original no quedó registrado. Eliminarlos descartaría casi el 13 % de la muestra y sesgaría la duración hacia arriba. La media y la mediana global llevan a todos los faltantes al mismo año y reducen la dispersión en más de 6 %. La mediana **por año de ingreso a la carrera actual** usa una variable observada que coincide con el año de origen en el 91 % de los registros válidos y es la que menos deforma la distribución (cambio de la desviación de +2,8 % frente a -6,3 % y -6,6 %). Es la estrategia elegida; la bandera anio_ing_carr_ori_imputada permite excluir esos registros en un análisis de sensibilidad.

In [10]:

```
df2, cifras = limpieza.imputar_mediana(df1, "anio_ing_carr_ori",
grupo="anio_ing_carr_act")

registrar("F2 · limpieza", f"anio_ing_carr_ori: {cifras['n_imputados'],} nulos
({cifras['pct']} %) imputados con {cifras['valor']}; "
        f"cambio en la desviación estándar {cifras['cambio_desv_pct']:+.2f} %;
bandera anio_ing_carr_ori_imputada")
```

```
df2, cifras_s = limpieza.imputar_mediana(df2, "sem_ing_carr_ori")

registrar("F2 · limpieza", f"sem_ing_carr_ori: {cifras_s['n_imputados']:,} nulos
({cifras_s['pct']} %) imputados con la mediana = {cifras_s['valor']:.0f}")

# Nulos residuales en columnas que NO son de análisis: se conservan como texto
(nombre_titulo / nombre_grado son descriptivos)

print(df2[["nombre_titulo", "nombre_grado", "cod_carrera", "version",
"mrun"]].isna().sum().to_string())

registrar("F2 · limpieza", "nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se
conservan sin imputar: son etiquetas de texto, no variables de análisis; "

"mrun con 7 nulos se conserva porque el análisis no cuenta personas sino
títulos")
```

Out:

```
[F2 · limpieza] anio_ing_carr_ori: 13,396 nulos (12.75 %) imputados con mediana por
anio_ing_carr_act; cambio en la desviación estándar +2.82 %; bandera
anio_ing_carr_ori_imputada

[F2 · limpieza] sem_ing_carr_ori: 13,396 nulos (12.75 %) imputados con la mediana = 1

nombre_titulo      2997
nombre_grado       8434
cod_carrera         297
version            287
mrun                7

[F2 · limpieza] nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se conservan sin
imputar: son etiquetas de texto, no variables de análisis; mrun con 7 nulos se conserva
porque el análisis no cuenta personas sino títulos
```

Paso 8 · Transformación por tipo de variable

Cada rol exige un tratamiento distinto: ordinales con **orden declarado por el equipo**, nominales con one-hot, alta cardinalidad agrupada, fechas parseadas y derivadas.

In [11]:

```
df3, mapa_edad = transformacion.codificar_ordinal(df2, "rango_edad",
transformacion.ORDEN_RANGO_EDAD)

df3, mapa_nivel = transformacion.codificar_ordinal(df3, "nivel_carrera_1",
transformacion.ORDEN_NIVEL_CARRERA)

print("rango_edad →", mapa_edad); print("nivel_carrera_1 →", mapa_nivel)

fuera = df3.attrs.get("rango_edad_fuera_de_orden", [])

n_fuera = int(df3["rango_edad_ord"].isna().sum())

registrar("F2 · transformación", f"ordinales codificadas con orden declarado:
rango_edad (6 niveles) y nivel_carrera_1 (4 niveles); "

f"{n_fuera} registros con rango_edad {fuera} quedan como faltante")
```

Out:

```
rango_edad → {'15 a 19 Años': 0, '20 a 24 Años': 1, '25 a 29 Años': 2, '30 a 34 Años':
3, '35 a 39 Años': 4, '40 y más años': 5}

nivel_carrera_1 → {'Técnico de Nivel Superior': 0, 'Licenciatura No Conducente a
Título': 1, 'Profesional Sin Licenciatura': 2, 'Profesional Con Licenciatura': 3}

[F2 · transformación] ordinales codificadas con orden declarado: rango_edad (6 niveles)
y nivel_carrera_1 (4 niveles); 0 registros con rango_edad [] quedan como faltante
```

In [12]:

```
# Si hubiera 'Sin Información' en rango_edad, se recupera desde la edad calculada
con las fechas (derivación), no se inventa

df3 = transformacion.derivar_fechas(df3)

print(df3[["edad_titulacion", "mes_titulacion"]].describe().round(1).to_string())
```

```

sin_info = df3["rango_edad_ord"].isna()
bins = [0, 20, 25, 30, 35, 40, 200]
df3.loc[sin_info, "rango_edad_ord"] = pd.cut(df3.loc[sin_info, "edad_titulacion"],
bins=bins, right=False, labels=False)
df3["rango_edad_ord"] = df3["rango_edad_ord"].astype(int)
registrar("F2 · transformación", f"fechas: fecha_obtencion_titulo y fec_nac_alu
parseadas; derivadas mes_titulacion y edad_titulacion "
f"(mediana {df3.edad_titulacion.median():.1f} años); {int(sin_info.sum())}
'Sin Información' de rango_edad recuperados desde edad_titulacion")

```

Out:

	edad_titulacion	mes_titulacion
count	105060.0	105060.0
mean	28.0	6.4
std	6.4	3.9
min	19.2	1.0
25%	24.1	3.0
50%	25.6	6.0
75%	29.1	10.0
max	80.9	12.0

[F2 · transformación] fechas: fecha_obtencion_titulo y fec_nac_alu parseadas; derivadas mes_titulacion y edad_titulacion (mediana 25.6 años); 0 'Sin Información' de rango_edad recuperados desde edad_titulacion

In [13]:

```

NOMINALES = ["tipo_inst_2", "jornada", "modalidad", "area_conocimiento",
"region_sede"]
df4, cols_onehot = transformacion.one_hot(df3, NOMINALES)
grupos = {c: [k for k in cols_onehot if k.startswith(c + "_")] for c in NOMINALES}
print({c: len(v) for c, v in grupos.items()})
registrar("F2 · transformación", f"one-hot encoding de {len(NOMINALES)} nominales →
{len(cols_onehot)} columnas 0/1")

df4, n_raras = transformacion.agrupar_raras(df4, "nomb_carrera", min_frec=100)
registrar("F2 · transformación", f"nomb_carrera: {n_raras:,} categorías con < 100
registros agrupadas en 'OTRA' → "
f"{df4['nomb_carrera_agrupada'].nunique()} categorías")

df4 = transformacion.binaria_01(df4, "gen_alu", 2, "mujer")
registrar("F2 · transformación", f"gen_alu recodificada como mujer (0/1):
{df4.mujer.mean():.1%} mujeres")

```

Out:

```

{'tipo_inst_2': 2, 'jornada': 5, 'modalidad': 3, 'area_conocimiento': 10,
'region_sede': 16}
[F2 · transformación] one-hot encoding de 5 nominales → 36 columnas 0/1
[F2 · transformación] nomb_carrera: 968 categorías con < 100 registros agrupadas en
'OTRA' → 129 categorías
[F2 · transformación] gen_alu recodificada como mujer (0/1): 56.3% mujeres

```

Paso 9 · Escalamiento: comparar los tres

La exploración mostró valores extremos ($\approx 9\%$ de atípicos en `dur_total_carr`) y la limpieza imputó con mediana. Por coherencia, el escalador debe centrar en la mediana, no en la media.

In [14]:

```
comp_esc = escalado.comparar_escaladores(df4["dur_total_carr"])
print(comp_esc.to_string())
fig, axes = plt.subplots(1, 3, figsize=(10, 3.2))
for ax, (nombre, f) in zip(axes, [("estandar", escalado.escalar_estandar),
("minmax", escalado.escalar_minmax), ("robusto", escalado.escalar_robusto)]):
    ax.hist(f(df4["dur_total_carr"]), bins=40); ax.set_title(f"dur_total_carr · {nombre}", fontsize=9)
guardar_figura("f2_02_escaladores")
```

Out:

	media	desv	mediana	ric	min	max
original	9.074	2.440	10.000	2.000	1.000	24.000
estandar	-0.000	1.000	0.380	0.820	-3.309	6.116
minmax	0.351	0.106	0.391	0.087	0.000	1.000
robusto	-0.463	1.220	0.000	1.000	-4.500	7.000

figura guardada: docs/figuras/f2_02_escaladores.png

In [15]:

```
ESCALAR = ["dur_total_carr", "dur_estudio_carr", "edad_titulacion"]
for c in ESCALAR:
    df4[f"{c}_esc"] = escalado.escalar_robusto(df4[c])
registrar("F2 · escalamiento", f"RobustScaler (mediana 0, RIC 1) aplicado a {ESCALAR}: coherente con la imputación por mediana y con los "
        f"atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho por el máximo de 24 semestres")
```

Out:

[F2 · escalamiento] RobustScaler (mediana 0, RIC 1) aplicado a ['dur_total_carr', 'dur_estudio_carr', 'edad_titulacion']: coherente con la imputación por mediana y con los atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho por el máximo de 24 semestres

Paso 10 · Validación: demostrar, no afirmar

Aserciones sobre el dataset final, comprobación de la propiedad de cada escalador y pruebas de las funciones (caso normal, límite y excepción).

In [16]:

```
COLS_ANALISIS = ["dur_total_carr", "dur_estudio_carr", "dur_proceso_tit",
"edad_titulacion", "mujer",
                "rango_edad_ord", "nivel_carrera_1_ord", "anio_ing_carr_ori",
"anio_ing_carr_ori_imputada"] + cols_onehot + [f"{c}_esc" for c in ESCALAR]
checks = validacion.validar_dataset_final(df4, COLS_ANALISIS, grupos,
n_filas_esperado=N0 - n_dup)
for c in checks: print("OK", c)
for c in ESCALAR:
    assert validacion.verificar_propiedad_escalador(df4[f"{c}_esc"], "robusto"), c
print("OK propiedad del escalador robusto verificada en", ESCALAR)
registrar("F2 · validación", f"{len(checks)} comprobaciones superadas sobre {len(COLS_ANALISIS)} columnas de análisis; propiedad mediana 0 / RIC 1 verificada")
```

Out:

OK sin nulos en columnas de análisis
OK todas las columnas de análisis son numéricas
OK cada grupo one-hot suma 1 por fila (5 grupos)
OK sin filas duplicadas

```
OK se conservó el número de filas esperado (105,060)
```

```
OK propiedad del escalador robusto verificada en ['dur_total_carr', 'dur_estudio_carr', 'edad_titulacion']
```

```
[F2 · validación] 5 comprobaciones superadas sobre 48 columnas de análisis; propiedad mediana 0 / RIC 1 verificada
```

```
In [17]:
```

```
import subprocess

res = subprocess.run([sys.executable, "tests/test_pipeline.py"],
capture_output=True, text=True)

print(res.stdout[-1500:]); assert res.returncode == 0, res.stderr

registrar("F2 · validación", "pruebas unitarias de src/ (tests/test_pipeline.py): "
+ res.stdout.strip().splitlines()[-1])
```

```
Out:
```

```
OK      test_atipicos_ric_detecta_extremo
OK      test_atipicos_serie_vacia
OK      test_codigo_a_nulo_normal
OK      test_columna_sin_nulos_no_imputa_nada
OK      test_eliminar_duplicados_sin_duplicados
OK      test_escaladores_cumplen_propiedad
OK      test_excepciones
OK      test_imputar_mediana_crea_bandera
OK      test_one_hot_suma_uno
OK      test_ordinal_respeto_orden_declarado
OK      test_una_sola_categoria_one_hot
```

```
11/11 pruebas superadas
```

```
[F2 · validación] pruebas unitarias de src/ (tests/test_pipeline.py): 11/11 pruebas superadas
```

Paso 11 · Persistencia y bitácora

El resultado va a data/processed/, nunca sobre data/raw/. Se relea para comprobar forma y columnas, y se exporta la bitácora.

```
In [18]:
```

```
SALIDA = Path("data/processed/titulados_2025_pregrado_univ_limpio.csv")
df4.to_csv(SALIDA, sep=";", index=False, encoding="utf-8")
chk = pd.read_csv(SALIDA, sep=";", low_memory=False)
assert chk.shape == df4.shape and list(chk.columns) == list(df4.columns)

print(f"Guardado y verificado: {SALIDA} → {chk.shape[0]:,} filas × {chk.shape[1]} columnas · {SALIDA.stat().st_size/1_048_576:.1f} MB")

registrar("F2 · persistencia", f"{SALIDA.name}: {chk.shape[0]:,} × {chk.shape[1]}, releído y verificado (forma y columnas)")

exportar("docs/bitacora.md")

print(Path("docs/bitacora.md").read_text(encoding="utf-8"))
```

```
Out:
```

```
Guardado y verificado: data/processed/titulados_2025_pregrado_univ_limpio.csv → 105,060 filas × 84 columnas · 66.7 MB
```

```
[F2 · persistencia] titulados_2025_pregrado_univ_limpio.csv: 105,060 × 84, releído y verificado (forma y columnas)
```

```
| Bitácora exportada a docs/bitacora.md (18 decisiones)
| # Bitácora de decisiones

| Generada el 2026-09-12 20:03

| - [F2 · obtención] lectura de data/processed/titulados_2025_pregrado_univ.csv → 105,063
|   filas × 40 columnas (sep ';', UTF-8)
|
| - [F2 · exploración] código 1900 en anio_ing_carr_ori: 13,396 registros (12,8 %) con
|   duración mediana 6 vs 10 semestres → no es un faltante aleatorio
|
| - [F2 · exploración] dur_total_carr: mediana 10 semestres, RIC 2, 9,845 atípicos (9.4
|   %) por regla 1,5·RIC → hay valores extremos
|
| - [F2 · limpieza] duplicados exactos: 3 filas eliminadas (0.003 %) → 105,060 filas
|
| - [F2 · limpieza] columnas constantes eliminadas (un solo valor tras el filtro de F1):
|   ['cat_periodo', 'tipo_inst_1', 'nivel_global']
|
| - [F2 · limpieza] anio_ing_carr_ori: código 1900 tratado como faltante en 13,396 filas
|   (12.8 %); sem_ing_carr_ori: código 0 tratado como faltante en 13,396 filas (12.8 %)
|
| - [F2 · limpieza] anio_ing_carr_ori: 13,396 nulos (12.75 %) imputados con mediana por
|   anio_ing_carr_act; cambio en la desviación estándar +2.82 %; bandera
|   anio_ing_carr_ori_imputada
|
| - [F2 · limpieza] sem_ing_carr_ori: 13,396 nulos (12.75 %) imputados con la mediana = 1
|
| - [F2 · limpieza] nombre_titulo (2,9 % nulos) y nombre_grado (8,0 %) se conservan sin
|   imputar: son etiquetas de texto, no variables de análisis; mrun con 7 nulos se conserva
|   porque el análisis no cuenta personas sino títulos
|
| - [F2 · transformación] ordinales codificadas con orden declarado: rango_edad (6
|   niveles) y nivel_carrera_1 (4 niveles); 0 registros con rango_edad [] quedan como
|   faltante
|
| - [F2 · transformación] fechas: fecha_obtencion_titulo y fec_nac_alu parseadas;
|   derivadas mes_titulacion y edad_titulacion (mediana 25.6 años); 0 'Sin Información' de
|   rango_edad recuperados desde edad_titulacion
|
| - [F2 · transformación] one-hot encoding de 5 nominales → 36 columnas 0/1
|
| - [F2 · transformación] nomb_carrera: 968 categorías con < 100 registros agrupadas en
|   'OTRA' → 129 categorías
|
| - [F2 · transformación] gen_alu recodificada como mujer (0/1): 56.3% mujeres
|
| - [F2 · escalamiento] RobustScaler (mediana 0, RIC 1) aplicado a ['dur_total_carr',
|   'dur_estudio_carr', 'edad_titulacion']: coherente con la imputación por mediana y con
|   los atípicos detectados; MinMax comprimiría el 90 % de los datos en un rango estrecho
|   por el máximo de 24 semestres
|
| - [F2 · validación] 5 comprobaciones superadas sobre 48 columnas de análisis; propiedad
|   mediana 0 / RIC 1 verificada
|
| - [F2 · validación] pruebas unitarias de src/ (tests/test_pipeline.py): 11/11 pruebas
|   superadas
|
| - [F2 · persistencia] titulados_2025_pregrado_univ_limpio.csv: 105,060 × 84, releído y
|   verificado (forma y columnas)
```

Resumen del pipeline

Paso	Entrada → salida	Decisión clave
Obtención	subconjunto F1 (105.063 × 40)	dato crudo intacto
Exploración	perfil de faltantes, categorías, atípicos	código 1900 no es un año: es faltante
Limpieza	−3 duplicados, −3	mediana por grupo (menor deformación de la dispersión) +

Paso	Entrada → salida	Decisión clave
	constantes, 1900/0 → NaN	bandera
Transformación	ordinales, one-hot, agrupación, fechas	orden declarado explícitamente; 'Sin Información' recuperado desde la edad
Escalamiento	3 duraciones/edad escaladas	RobustScaler, coherente con mediana y atípicos
Validación	5 asserts + 11 pruebas	todo verificado, no afirmado
Persistencia	data/ processed/..._limpio. csv + docs/bitacora.md	relectura comprobada